CO-2Assignment

Name: G R Aakaash **Course Code:** CSA0201

Reg no: 192412068 **Course Name:** C-Programming

TASK-1

CODE:

```
#include <stdio.h>

int main()
{
   double price, gross, discount, tax, total, net;
   int qty, senior, count = 0;

   scanf("%lf%d%d", &price, &qty, &senior);

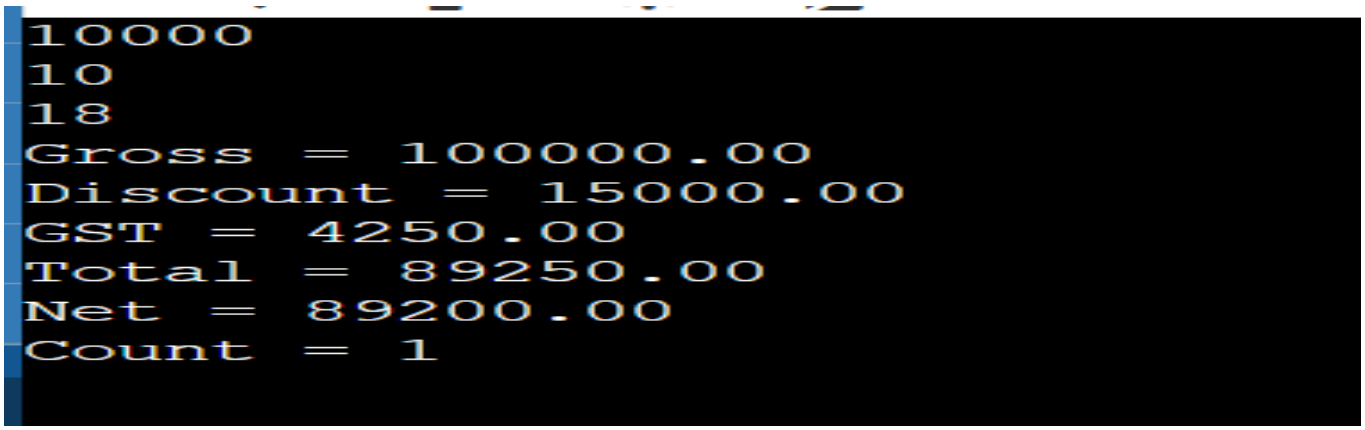
   gross = price * qty;
   discount = senior ? gross * 15.0 / 100 : 0;
   tax = (gross - discount) * 5.0 / 100;
   total = gross - discount + tax;

   net = (total > 1000) ? total - 50 : total;
   count = count + 1;

   printf("Gross = %.2lf\n", gross);
   printf("Discount = %.2lf\n", discount);
   printf("GST = %.2lf\n", tax);
   printf("Total = %.2lf\n", total);
   printf("Net = %.2lf\n", net);
   printf("Count = %d\n", count);

   return 0;
}
```

OUTPUT:



TASK-2

CODE:

```
#include <stdio.h>

int calls = 0;

int verify_card()
{
   calls++;
   return 1;
}

int main()
{
   int senior, eligible;

   for (int i = 0; i < 1000; i++)
   {
      senior = (i % 2 == 0);
      eligible = (i % 4 == 0);

      if ((senior && eligible) || verify_card())
      {
      }
   }

   printf("Short circuit calls = %d\n", calls);

   calls = 0;
```

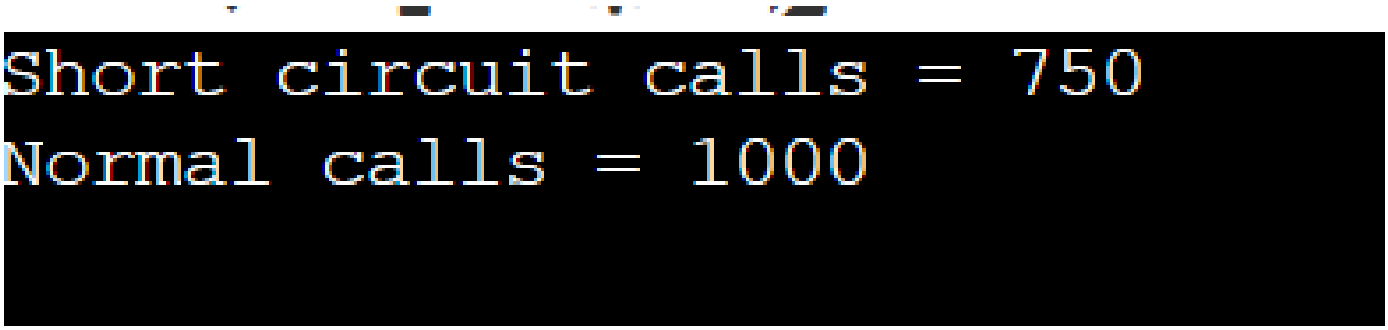
```
    for (int i = 0; i < 1000; i++)
    {
        senior = (i % 2 == 0);
        eligible = (i % 4 == 0);

        if ((senior & eligible) | verify_card())
        {
        }
    }

    printf("Normal calls = %d\n", calls);

    return 0;
}
```

OUTPUT:



TASK-3

CODE:

```
#include <stdio.h>

enum {LIGHT=0, FAN=1, AC=2, HEATER=3, PUMP=4};

unsigned char switch_on(unsigned char reg, int dev)
{
    return reg | (1u << dev);
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return reg & ~(1u << dev);
}
```

```
unsigned char toggle(unsigned char reg, int dev)
{
   return reg ^ (1u << dev);
}

int is_on(unsigned char reg, int dev)
{
   return (reg & (1u << dev)) != 0;
}

void show(unsigned char reg)
{
   for(int bit=7; bit>=0; bit--)
      putchar(((reg >> bit) & 1u) ? '1' : '0');

   putchar('\n');
}

int main(void)
{
   unsigned char reg=0;

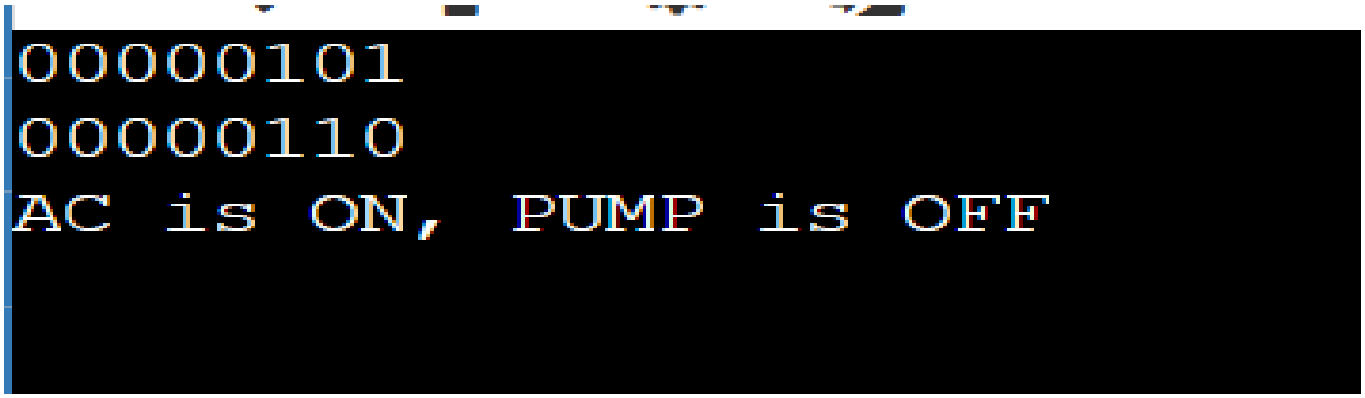
   reg=switch_on(reg,LIGHT);
   reg=switch_on(reg,AC);
   show(reg);

   reg=toggle(reg,FAN);
   reg=switch_off(reg,LIGHT);
   show(reg);

   printf("AC is %s, PUMP is %s\n",
   is_on(reg,AC)?"ON":"OFF",
   is_on(reg,PUMP)?"ON":"OFF");

   return 0;
}
```

OUTPUT:



TASK-4

CODE:

```
#include <stdio.h>

char grade(int marks)
{
   if (marks < 0 || marks > 100)
      return '?';

   switch (marks / 10)
   {
      case 10:
      case 9:
         return 'S';

      case 8:
         return 'A';

      case 7:
         return 'B';

      case 6:
         return 'C';

      case 5:
         return 'D';

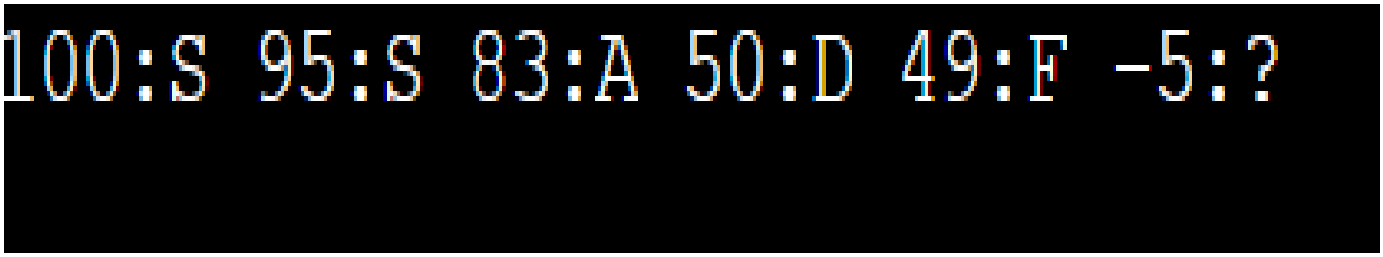
      default:
         return 'F';
   }
}

int main()
{
   int marks[] = {100, 95, 83, 50, 49, -5};

   for(int i=0; i<6; i++)
      printf("%d:%c ", marks[i], grade(marks[i]));

   return 0;
}
```

OUTPUT:



TASK-5

CODE:

```
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n)
{
   checks = 0;

   if(n < 2)
      return 0;

   for(unsigned long i=2; i<=n/i; i++)
   {
      checks++;

      if(n % i == 0)
         return 0;
   }

   return 1;
}

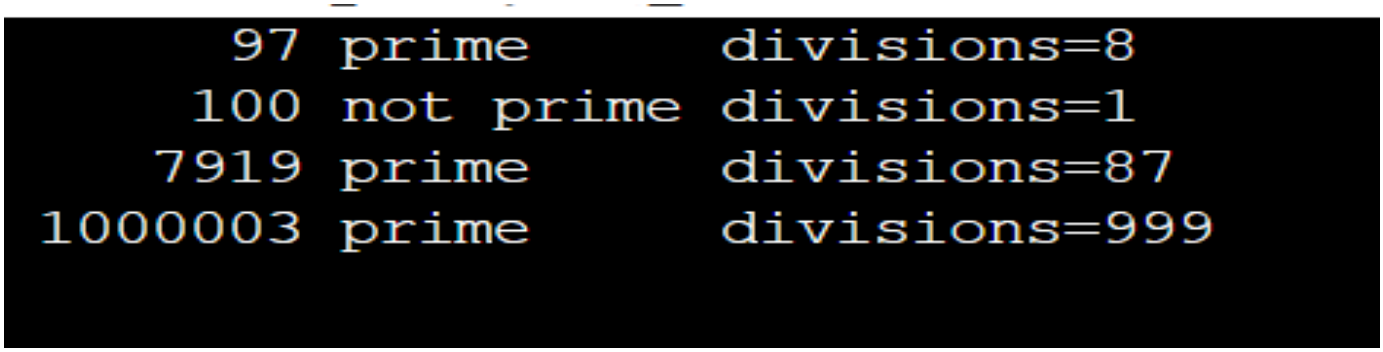
int main()
{
   unsigned long tickets[] = {97, 100, 7919, 1000003};

   for(int k=0; k<4; k++)
   {
      int p = is_prime(tickets[k]);

      printf("%8lu %-9s divisions=%lu\n",
      tickets[k], p ? "prime" : "not prime", checks);
   }
```

```
    return 0;
}
```

OUTPUT:



TASK-5 (1)

CODE:

```
#include <stdio.h>

int main()
{
    unsigned long n, i=2;
    int prime=1, checks=0;

    scanf("%lu", &n);

    if(n < 2)
        prime=0;

    while(prime && i<=n/i)
    {
        checks++;

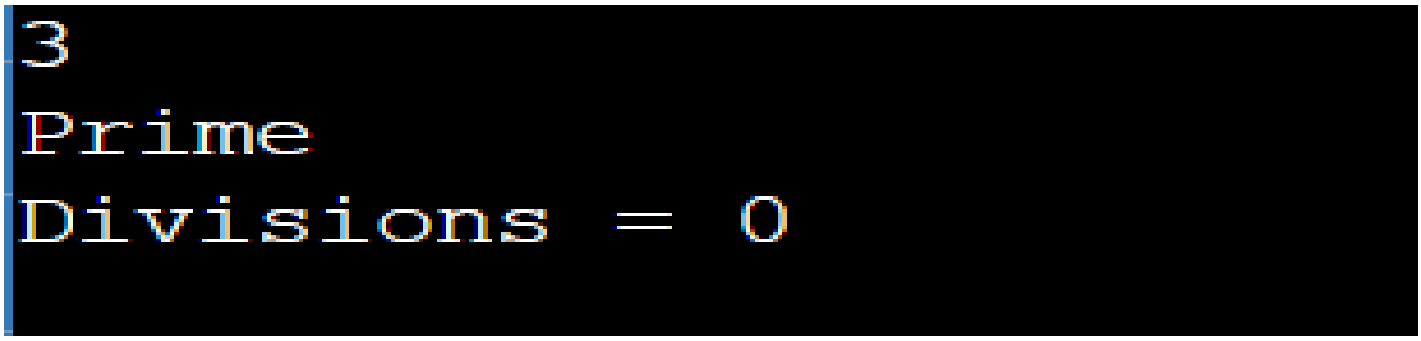
        if(n%i==0)
            prime=0;

        i++;
    }

    printf("%s\n", prime ? "Prime" : "Not Prime");
    printf("Divisions = %d\n", checks);

    return 0;
}
```

OUTPUT:



TASK-5

CODE:

```
#include <stdio.h>

int main()
{
   unsigned long n, i=2;
   int prime=1, checks=0;

   scanf("%lu", &n);

   if(n < 2)
      prime=0;

   if(prime)
   {
      do
      {
         if(i > n/i)
            break;

         checks++;

         if(n%i==0)
         {
            prime=0;
            break;
         }

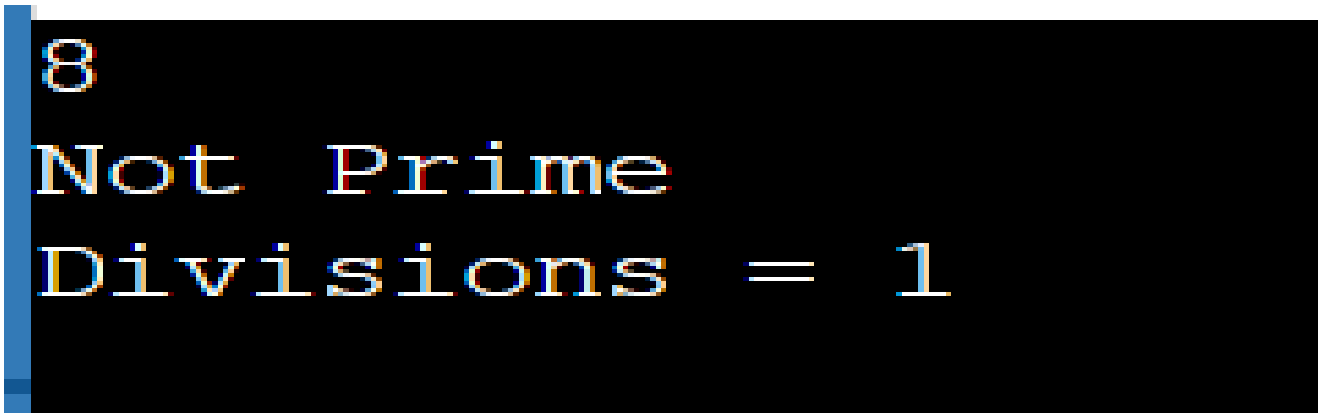
         i++;

      } while(1);
   }

   printf("%s\n", prime ? "Prime" : "Not Prime");
   printf("Divisions = %d\n", checks);
```

```
    return 0;
}
```

OUTPUT:



TASK-5

CODE:

```
#include <stdio.h>

int main()
{
    int n=100;
    int prime[101];

    for(int i=0; i<=n; i++)
        prime[i]=1;

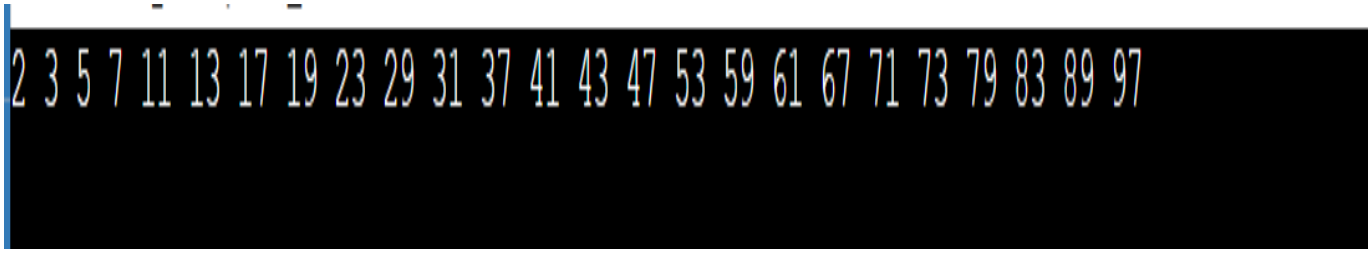
    prime[0]=prime[1]=0;

    for(int i=2; i*i<=n; i++)
    {
        if(prime[i])
        {
            for(int j=i*i; j<=n; j+=i)
                prime[j]=0;
        }
    }

    for(int i=2; i<=n; i++)
    {
        if(prime[i])
            printf("%d ", i);
    }
```

```
    return 0;
}
```

OUTPUT:



TASK-6

CODE

```
#include <stdio.h>

int main()
{
    double balance, rate, repayment;
    double interest, total_interest=0;
    int month=0;

    scanf("%lf%lf%lf", &balance, &rate, &repayment);

    while(balance > 0.01)
    {
        if(month >= 360)
        {
            printf("Maximum period reached\n");
            break;
        }

        interest = balance * rate / 100;
        total_interest += interest;
        balance += interest;

        if(repayment <= interest)
        {
            printf("Repayment is insufficient\n");
            break;
        }
```

```
        double payment = repayment;

        if(payment > balance)
            payment = balance;

        balance -= payment;
        month++;

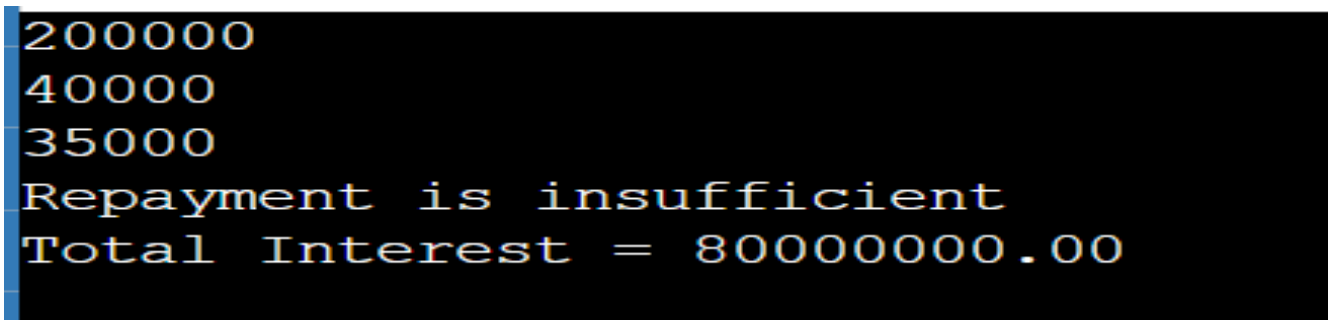
        printf("Month %d: Balance = %.2lf\n", month, balance);
    }

    if(balance <= 0.01)
        printf("Loan completed in %d months\n", month);

    printf("Total Interest = %.2lf\n", total_interest);

    return 0;
}
```

OUTPUT:



TASK-7

CODE:

```
#include <stdio.h>

int main()
{
    unsigned long n, i;
    unsigned long checks=0;
    int prime=1;
```

```
    scanf("%lu", &n);

    if(n < 2)
        prime=0;

    else if(n == 2)
        prime=1;

    else if(n % 2 == 0)
    {
        checks++;
        prime=0;
    }
    else
    {
        for(i=3; i<=n/i; i+=2)
        {
            checks++;

            if(n%i==0)
            {
                prime=0;
                break;
            }
        }
    }

    if(prime)
        printf("%lu is Prime\n", n);
    else
        printf("%lu is Not Prime\n", n);

    printf("Number of divisions = %lu\n", checks);

    return 0;
}
```

OUTPUT:

